Verification guide

Rank-six categorification through multiplicity eight

23 September 2026

1 Scope and authoritative records

This guide accompanies *Rank-six fusion rings: categorification criteria and models through multiplicity eight*. The counting input has 16612 rings, with exact-multiplicity counts $39, 154, 384, 872, 1582, 2768, 3814, 6999$. It comes from the public *Fusion Ring Census*; no independent census is performed here.

The classification ledger contains 16511 general exclusions, 53 unitary-positive rings, one spherical positive ring with no pseudo-unitary categorification, and 47 residuals. The 54 positive rings have 61 constructor records, including alternative models. General categorifiability is completely classified through multiplicity two: precisely 43 of the 193 rings in that range are categorifiable, all have spherical models, and 42 have unitary models. The exception to pseudo-unitary existence is Schopieray's $R^{(6)}_{2,124}$.

The authoritative files are `data/census.json`, `data/decisions.json` and `data/classification_counts.json`. The CSV ledger and manuscript tables are derived exports. The model catalogue records sources, categorical structures and explicit basis permutations; the residual export contains complete tensors and all five existence fields. A structure-specific exclusion does not change an unknown general existence field into a general exclusion.

2 Frozen census input

The source is *Fusion Ring Census*, revision

`b86eb6dc6eeeafe5fc5f6906d7b074e0322e9726`.

The rank-six files are `results/census.json` and `results/rank6/tables.txt.gz`. The compressed tensor SHA-256 is

`0138c6228dbb06b0861ad38b8dfe67fcd1fc829add80d441c34c09a1afb78675`.

The uncompressed tensor SHA-256 is

`7898a833dfffd0f645436c1d326740db29ea6a29980de307c67227f0b67122b0`.

The bundled tensor file is `data/rank6_tables.txt.gz`; `provenance/repository_source.json` records its source and hashes. Import, classification and checking run offline from these files. The pinned source supplies the completeness claim. Import checks fix the input and its correspondence with the classification registry; they do not re-enumerate rings or turn a hash into a completeness proof. The established representatives and identifiers through multiplicity seven are preserved. Exact-multiplicity-eight representatives are sorted in the same row-major convention. Every obstruction and model comparison operates on full tensors, never on an identifier alone.

3 Automated classification and exact checking

The manuscript orders exclusion families to expose inexpensive structural and arithmetic tests before larger finite searches. Its sequential table counts each removal once and reports what remains at every exact multiplicity. This order balances observed usefulness with transparency; it is not a proof of a universally optimal runtime. The first unresolved multiplicity identifies the next stratum requiring work. A zero residual count at some larger tested multiplicity says nothing by itself about untested multiplicities. The dimension-order and character-induction stages first contribute new top-level removals at multiplicity eight, one and four respectively. They may occur inside earlier nested certificates, and their first contribution cannot establish criterion stability beyond multiplicity eight.

Use Python with the versions in `requirements.txt`, a C++17 compiler for the finite-search helpers, and `pdflatex` for the PDFs. Native helpers are compiled from their supplied source as needed. The verification and discovery entry points select SymPy's pure-Python polynomial backend before importing SymPy. The archived exact separating vectors are indexed by the resulting equation-row order, including when the optional python-flint package is installed elsewhere. The verification commands are:

```
python -m pip install -r requirements.txt
python check_manifest.py
python verify.py --jobs 2 --out verification/current/recheck.jsonl
python -O verify.py --jobs 2 --out verification/current/recheck_O.jsonl
python software/current_regressions.py
```

To repeat ordered discovery on the source tensors, use:

```
python software/import_repository.py
python software/classify.py --jobs 4
```

Reports are stored in `verification/classification/`. The files `stages.json` and `summary.json` record actual stage counts and outcomes. The adjacent `attempts.jsonl` distinguishes failed or unfinished attempts from accepted certificates. Discovery can be more expensive than checking existing certificates. These commands apply categorification criteria; they do not enumerate new fusion rings.

Run the manifest check before writing reports: a new report changes file bytes. The normal and optimized Python modes check that the mathematical checks do not depend on removable assertions. A failed subprocess must have a nonzero return code and cannot be reported as a successful check. The saved reports state their input, scope and actual outcomes; an older report does not establish a result for a larger census.

A negative certificate is accepted only after its checker reconstructs the required number fields, finite alternatives, separating witnesses and exact inequalities from the original tensor. Numerical linear and mixed-integer optimization may assist discovery, but their entry points are disabled during negative-certificate checking. A timeout, ineligibility, numerical infeasibility flag or missing certificate leaves the case unresolved. A successful necessary relaxation also leaves it unresolved unless a categorical construction is available.

The option `--proofs-only` retains source-integrity checks but omits positive-model reconstruction; it is not a complete package check. Regression controls cover native/reference agreement and deliberately corrupted certificates. Saved verification is computer-assisted exact arithmetic, not proof-assistant formalization: the categorical input theorems and their hypotheses remain mathematical inputs to the argument.

4 Necessary criteria and their certificates

4.1 Dimension order and character constraints

`software/dimension_order.py` computes the $\mathbb{Z}$-span of simple dimensions in a rational power basis using Hermite normal form. The necessary condition is membership of the required codegree quotient in this actual order, not merely in the maximal order. The checker verifies the codegree root, lattice, separating functional and nonintegral value of the functional on the quotient. General wrappers exhaust all required oriented signed tables and dimension-character alternatives. Failure for only one possible character is not general nonexistence.

Character-aware induction uses joint eigenvalue labels, reciprocity and Galois restrictions in the same central-induction system. A discarded column must violate a proved necessary condition. Existing exact separating vectors remain valid after such columns are removed, because the inequalities on the retained columns are unchanged. The certificate allocation records the accepted wrapper as well as its inner tests.

4.2 Signed spherical traces

`software/signed_spherical_traces.py` reconstructs signed characters and low-column searches. It keeps integer restriction multiplicities, Gram positivity, signed indicator equalities, joint Galois closure and modular equations. It does not replace signed categorical dimensions with absolute values or import pseudo-unitary mass bounds without a valid reduction. The stored certificate for $R^{(6)}_{4,446}$ exhausts 290 low matrices, two surviving low frames, two character labellings per frame and all 32 second-indicator choices per labelling; its 128 separating vectors are checked exactly.

4.3 Module, subring and Morita reductions

Subring descent uses the published or cited categorification statements with their stated ranks, multiplicities and categorical assumptions. A larger counting census does not enlarge the range of a lower-rank categorification theorem. Module-Gram and Morita constraints likewise require the module or subcategory hypotheses proved in the manuscript. Their applicability is checked before a certificate is accepted.

5 Positive models

Positive conclusions require a categorical construction and a complete tensor match. Sources include quantum-group categories, finite-group representation categories, near-group and subfactor constructions, central-boson quotients, equivariantizations and relative centers. The model catalogue preserves its bibliographic attribution and the precise categorical structure supplied by each source.

The constructors in `software/residual_models.py` identify Extended Haagerup, Izumi's $C_5 + 5$ category, the mixed half-$E_6/PSU(2)_6$ quotient and sign-swap examples. The relative Fibonacci center of TT_3 is reconstructed in `software/relative_fibonacci_models.py`; its restriction matrix and exact linear system are stored in `data/relative_tt3.json`. The Fibonacci inputs and categorical relative-center construction are part of the argument. A relative center is not a Müger centralizer.

The constructors in `software/through7_models.py` reconstruct the Asaeda–Haagerup $\mathcal{AH}_1$ table and the representation ring of $(C_3 \times C_3) \rtimes Q_8$. The latter constructs the order-72 group, checks its character table and obtains tensor coefficients by exact character inner products. The quantum-group, cochain, finite-group and relative-center controls all compare complete multiplication tensors.

Centrality in a fusion ring is not a categorical central lift, and an associative formal tensor is not automatically categorifiable. A numerical dimension match supplies neither the tensor match nor a category. These distinctions apply to alternative models as well as to newly resolved rings.

6 Residual tensors and finite witnesses

The first unresolved stratum is read from `data/unresolved_tensors.json`. The tensor $R^{(6)}_{3,215}$ is the central low-multiplicity research target described in the manuscript. Its file `systems/integral_frontier_215.json` records a feasible point for a specified central-induction relaxation. The commands

```
python software/modular_frontier.py
python -O software/modular_frontier.py
```

reconstruct the low frame, character labels, second indicators, pruning and 230 rational equations in 199 variables. The supplied point has nonnegative integral coordinates and formal central rank 22. This statement concerns that relaxation; it supplies neither individual twists, a complete center fusion ring, nor associators.

6.1 The joint product test and literature comparisons

The first general residual remains $R^{(6)}_{3,215}$. The manuscript proves a joint fourth-order product-Gram condition for pseudo-unitary center data with specified self-dual low objects. This couples all low products in a single nonnegative integral Gram factorization and enforces their restriction equations. The checking command is

```
python research/m8_frontier/low_product_gram.py --check
```

It reconstructs the inherited induction witness, the cubic and fourth-order moments, all fifteen product restrictions and the expanded Gram factorization. The rank-22 formal point survives: the residual 15×15 Gram matrix has rank three, its three high restriction types have multiplicities $(4, 8, 4)$, and the forced per-object product coefficients are nonnegative integers. The test supplies no complete center fusion tensor, individual twists or associators. Rejecting just one proposed center point would not by itself exclude the ring either.

The two literature checks are

```
python research/m8_frontier/premodular_bound.py
python research/m8_frontier/nrw_rank6_scan.py
```

The first implements the rank-six bound $\mathrm{FPdim} \leq 168$ for braided spherical categorifications derived from the Dong–Schopieray catalogue. It checks the dimension character and the strict inequality $136 + 32\sqrt{17} > 168$ exactly for the frontier tensor. The result excludes premodular categorification only; it is not a general, pivotal, spherical, pseudo-unitary or unitary exclusion without the additional braiding hypothesis.

The second reads the frozen literal integer tensors in the Ng–Rowell–Wen rank-six ancillary data and screens all 16612 census tensors. Exact basis-permutation comparisons follow invariant filtering. The 192 modular records comprise 20 Galois orbits and eleven based-ring types, all of multiplicity at most two; there is no new model for the multiplicity-three frontier or the multiplicity-eight stratum. The source comparison does not exclude arbitrary fusion categories. Both programs save explicit scope and source identities with their reports. No ring-specific hand arithmetic is required to reproduce any of these calculations.

The feasible witnesses for $R_{2,4}^{(6)}$ are separate controls for a ring with an independently justified positive model. They must not be transferred to a residual tensor. Exact feasible points explain the limits of the corresponding necessary systems; they do not show that all conceivable refinements would fail. A literature comparison counts as a new model only when the source actually constructs a category and its full fusion tensor agrees.

7 Regenerating the presentation

The manuscript has seven source chapters organized by mathematical operation. The following commands rebuild the derived ledgers, tables and the two companion PDFs from the retained records:

```
python software/release.py
python software/model_catalog.py
python software/build_paper_tables.py
python build_papers.py
```

The model registry itself can be reconstructed with `python software/residual_models.py` before these commands. All generated counts must agree with the authoritative decision ledger. The eight OEIS terms count based rings rather than categorifiable rings; no accession or online submission is claimed. No census enumeration is part of this reproduction procedure.

AI assistance. GPT-6 Astra assisted with research, computation, and document preparation.